Alma Mater Studiorum · Università di Bologna

CORSO DI LAUREA IN INGEGNERIA E SCIENZE INFORMATICHE

Smart Drone Hangar

Relazione per il corso di Sistemi Embedded e IoT

Studente:
Riem Boukhama
0001129990

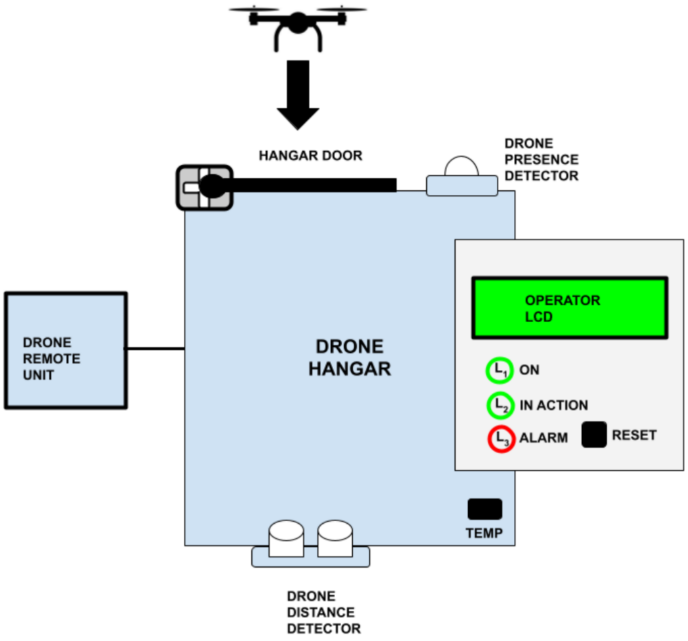
Anno Accademico 2025/2026

1 Introduzione

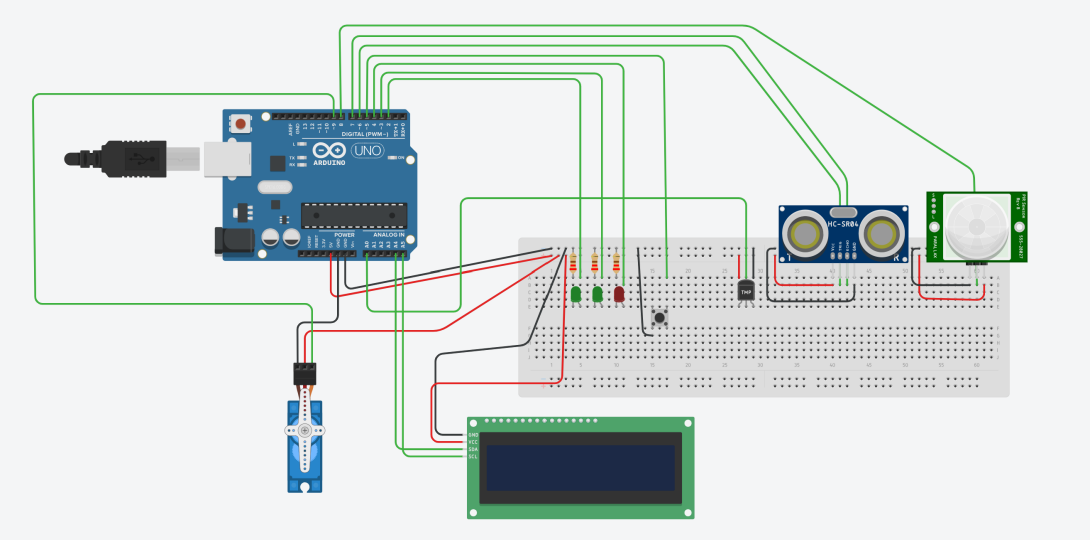
Il sistema Smart Drone Hangar è un'architettura progettata per la gestione automatizzata di un hangar per droni. L'obiettivo del progetto è fornire una piattaforma per il controllo remoto delle fasi di decollo e atterraggio del drone con la gestione dell'apertura e chiusura dell'hangar.

L'intera architettura del progetto è suddivisa in due macro-componenti principali:

- **L'Hangar:** Costituito da un sistema embedded basato su microcontrollore Arduino Uno. Il suo compito è gestire i sensori ambientali e l'attuatore (porta dell'hangar).
- **La Drone Remote Unit (DRU):** Un'applicazione eseguita su PC e sviluppata in linguaggio Python, dotata di un'interfaccia grafica. La DRU funge da pannello di controllo principale per l'operatore e simula il canale di comunicazione tra operatore e l'hangar.



1.1 Schema Hardware



Lato hardware sono stati usati i seguenti componenti:

- **Microcontrollore (Arduino Uno R3)**
- **Servomotore (HS-53)**
- **Sensore Sonar (HC-SR04)**
- **Sensore Movimento PIR (HC-SR501)**
- **Sensore Termico Analogico (TMP36)**
- **Display LCD I2C**
- **2 LED Verdi**
- **1 LED Rosso**
- **Pulsante tattile**

3 Architettura Software e Hardware

Il firmware in esecuzione sul microcontrollore Arduino Uno è stato sviluppato seguendo i paradigmi della programmazione orientata agli oggetti (C++) e strutturato secondo un'architettura gerarchica a livelli (Layer-based Architecture). Questo approccio garantisce modularità, permettendo di incapsulare le funzionalità e separare nettamente l'interazione con l'elettronica dalla logica decisionale del sistema.

Hardware Abstraction Layer (HAL)

Al livello più basso dell'architettura è stato implementato l'Hardware Abstraction Layer (HAL). Il suo scopo è astrarre gli aspetti tecnici di basso livello fornendo ai livelli superiori un'interfaccia indipendente dall'hardware. Ogni dispositivo fisico dell'hangar è stato modellato come una classe indipendente (es. Button, Led, Sonar, HangarDoor e TempSensor), separandone l'interfaccia nel file header e l'implementazione nel file sorgente.

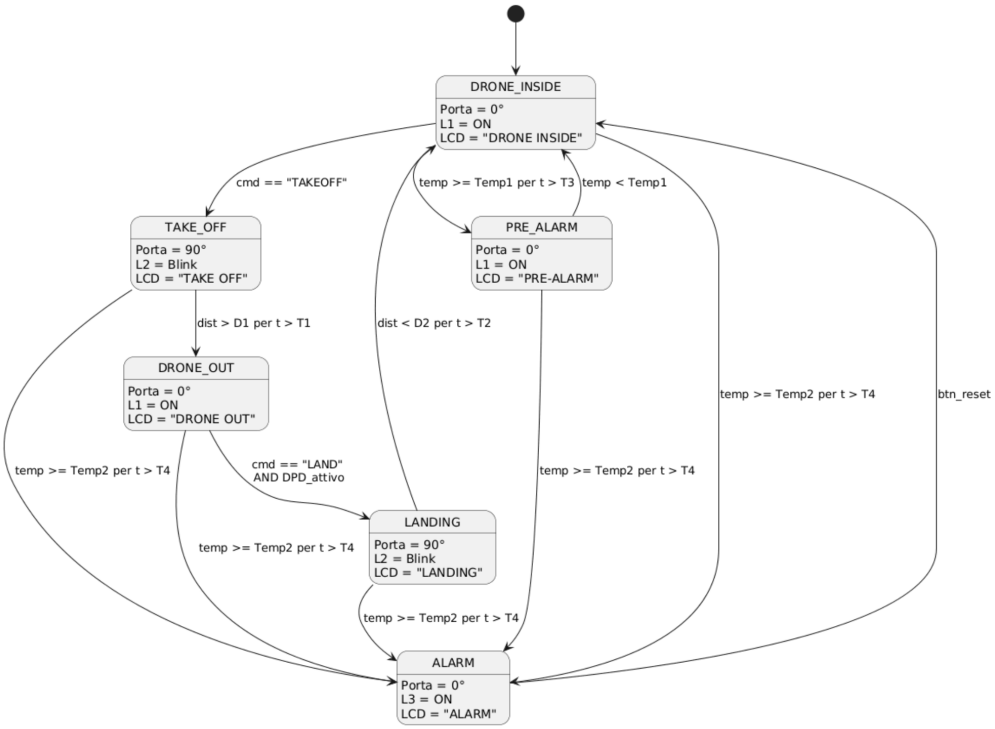
Schedulazione e Architettura a Task

Il comportamento del sistema è stato scomposto in una serie di compiti indipendenti, definiti Task, le cui esecuzioni si sovrappongono nel tempo. Ogni task (come ad esempio HangarTask, TempTask, Serial Task e Blink Task) eredita da una classe base astratta, la quale definisce l'interfaccia standard e i metodi fondamentali `init()` e `tick()`. Il coordinamento e l'avanzamento dei task sono gestiti da uno Scheduler che alterna i task in modo circolare. Per mantenere il sistema reattivo, il firmware è stato progettato in modo asincrono: lo scheduler elimina l'impiego di funzioni temporali bloccanti, sfruttando invece la misurazione del tempo tramite la funzione `millis()`. Ad ogni ciclo, lo scheduler invoca la funzione `tick()` dei singoli task solo se il loro specifico periodo di attivazione è trascorso.

4 Modellazione del Comportamento: Macchine a Stati Finiti (FSM)

Il sistema è basato su una Macchina a Stati Finiti (FSM), la quale controlla periodicamente gli input (sensori fisici e comandi dalla DRU) per gestire i corrispondenti output del sistema. Il funzionamento dell'hangar è strutturato in stati operativi, all'interno dei quali viene definita la configurazione di ciascuna periferica. Il passaggio da uno stato all'altro avviene attraverso transizioni regolate da condizioni e controlli sul tempo: quando le condizioni di tempo e di input vengono verificate, il sistema esegue le relative azioni ed evolve verso il nuovo stato operativo.

Analisi degli Stati Operativi



La FSM comprende sei stati ben definiti:

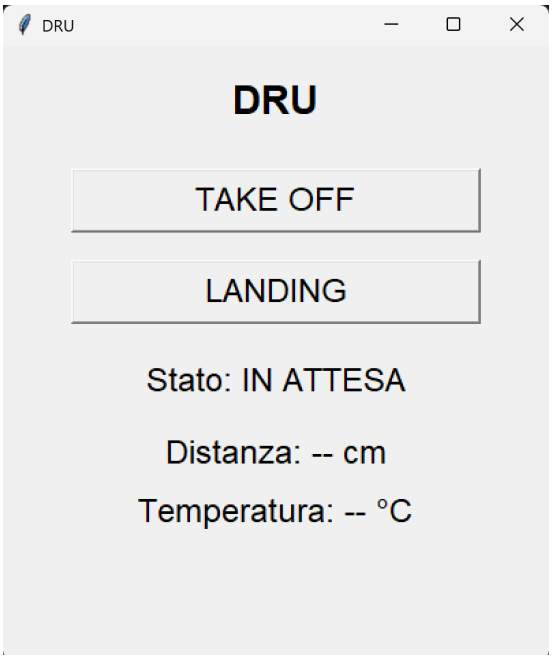
- **DRONE_INSIDE:** Rappresenta la condizione iniziale con il drone all'interno dell'hangar. La porta dell'hangar è completamente chiusa (servomotore a 0°), il led verde L1 è acceso a indicare la disponibilità dell'hangar, mentre L2 e L3 rimangono spenti. Il display LCD mostra il messaggio DRONE INSIDE. In questo stato, il sistema rimane in ascolto del comando di take-off.
- **TAKE_OFF:** é la fase di decollo e uscita del drone. All'ingresso nello stato, la porta viene sollevata a 90° e il LED giallo L2 inizia a lampeggiare con un intervallo di 0.5 secondi, mentre il display LCD mostra TAKE OFF. Il sistema monitora la distanza tramite il sensore sonar per verificare l'effettiva uscita del drone.
- **DRONE_OUT:** Rappresenta la fase in cui il drone è completamente uscito dall'hangar. La porta dell'hangar viene richiusa a 0° e il led L2 smette di lampeggiare, il led L1 rimane acceso e il display LCD mostra DRONE OUT. L'hangar rimane in attesa del messaggio di rientro (LANDING) da parte dell'operatore.

- **LANDING:** Rappresenta la fase di approccio e atterraggio del velivolo nell'hangar. La porta viene nuovamente aperta di 90°, il display mostra LANDING e il LED L2 inizia a lampeggiare. Il sensore sonar verifica in continuazione la distanza del drone per rilevare l'entrata nell'hangar, il drone si considera atterrato quando il sonar rileva una distanza minore di D2 per un tempo maggiore di T2.
- **PRE_ALARM:** Stato di preallarme in caso di un aumento della temperatura sopra la soglia temp1 per un tempo maggiore di T3. La porta dell'hangar resta chiusa, il LED L1 è acceso e lo schermo segnala PRE-ALARM. In questo stato le nuove richieste della DRU vengono bloccate mentre viene monitorato il ritorno della temperatura al di sotto della soglia temp1.
- **ALARM:** Stato di arresto del sistema in caso la temperatura raggiunga la soglia temp2 per un tempo maggiore di T4. La porta dell'hangar viene forzatamente chiusa, i LED L1 e L2 vengono spenti, il LED rosso L3 rimane acceso in modo fisso e il display LCD mostra ALARM. Tutte le azioni dell'hangar sono interrotte.

5 Drone Remote Unit (DRU)

La Drone Remote Unit (DRU) rappresenta il livello applicativo del sistema, fungendo da pannello di controllo a disposizione dell'operatore. L'applicazione è stata sviluppata in linguaggio Python, sfruttando la libreria tkinter per la gestione dell'interfaccia grafica e la libreria pyserial per la comunicazione su bus seriale USB con Arduino Uno.

Design dell'Interfaccia



L'interfaccia utente include due sezioni principali:

- **Controlli di Comando:** Due pulsanti dedicati all'invio dei comandi (TAKE OFF e LANDING)
- **Monitoraggio:** Tre etichette di testo dedicate rispettivamente allo stato della FSM (Stato), alla distanza in tempo reale rilevata dal sonar (Distanza) e alla temperatura rilevata all'interno dell'hangar (Temperatura).

Architettura Multi-Threading

L'architettura software separa la gestione grafica dalla gestione dei dati su due thread di esecuzione concorrenti tramite il modulo `threading`:

- **Thread principale:** Esegue il ciclo principale ad eventi della GUI (`root.mainloop()`).
- **Daemon Thread:** Esegue un loop in background assegnato alla funzione (`read()`) per verificare la presenza di dati nel buffer `ard.in_waiting`, all'arrivo di una stringa ne estrae i dati e aggiorna le etichette nell'interfaccia grafica.

6 Video Dimostrativo

Il video dimostrativo è disponibile al seguente link: assignment-02.mp4.